TRABAJO FIN DE GRADO
Desarrollo de Aplicaciones Multiplataforma

∼ FPConnect ∼

Plataforma web para conectar alumnos de FP,
centros educativos y empresas en Andalucía

Alumnos: Rafael Marcos, Jose Antonio Roda, Angel Amil Cobacho

Ciclo: Técnico Superior en Desarrollo de Aplicaciones Multiplataforma (DAM)

Curso: 2025 – 2026

Repositorio: https://github.com/rafamarcoss/FPCONNECT

Comunidad Autónoma de Andalucía · 2025

Índice general

1. Resumen **3**

2. Glosario (ES/EN) **4**

3. Planteamiento del problema y justificación **5**

3.1. Contexto del problema 5

3.2. Motivación del proyecto 5

4. Fundamentación teórica y estado del arte **6**

5. Objetivos **7**

6. Planificación temporal y evaluación de coste **8**

6.1. Planificación temporal 8

6.2. Evaluación de coste (estimación académica) 8

7. Análisis de requisitos **10**

7.1. Requisitos funcionales priorizados 10

7.2. Requisitos técnicos de seguridad y acceso 10

8. Análisis del Sistema **11**

8.1. Identificación de actores 11

8.2. Requisitos funcionales 11

8.2.1. Módulo de autenticación y gestión de roles 11

8.2.2. Funcionalidades del Alumno 11

8.2.3. Funcionalidades del Centro FP 12

8.2.4. Funcionalidades de la Empresa 12

8.3. Requisitos no funcionales 13

9. Diseño del Sistema **14**

9.1. Arquitectura general 14

9.2. Estructura de directorios 14

9.3. Modelo de datos 15

9.4. Diseño de la interfaz de usuario 16

10. Metodología **18**

11. Codificación **19**
11.1. Entorno de programación y herramientas 19
11.2. Aspectos relevantes de implementación 19
11.2.1. Validación de datos 19
11.2.2. Control de acceso 19
11.2.3. Protección de la información 19
11.2.4. Estructura modular del backend 19

12. Resultados **20**
12.1. Estado actual del desarrollo 20

13. Pruebas de ejecución **21**
13.1. Pruebas funcionales 21
13.2. Pruebas de usabilidad 21
13.3. Pruebas de accesibilidad 21
13.4. Pruebas de seguridad 21
13.5. Pruebas de carga (estado actual) 21

14. Manuales de usuario **22**
14.1. Objetivo 22
14.2. Requisitos e instalación 22
14.3. Operaciones habituales 22
14.4. Mensajes de error comunes 22

15. Conclusiones **23**
15.1. Grado de consecución de objetivos 23
15.2. Limitaciones 23
15.3. Futuras líneas de trabajo 23

16. Bibliografía **24**

17. Anexos **25**

1. Resumen

FPConnect es un proyecto intermodular orientado al desarrollo de una plataforma digital para conectar a los tres agentes principales de la Formación Profesional: alumnado, centros educativos y empresas. La solución se plantea como una aplicación web full-stack con arquitectura cliente-servidor, roles diferenciados y servicios en tiempo real para mejorar la comunicación y la empleabilidad en el entorno de FP.

La implementación técnica se basa en un frontend con React y Vite, y un backend con Node.js y Express, utilizando PostgreSQL como sistema gestor de base de datos y Prisma como ORM. La autenticación se resuelve con JWT y refresh tokens, y la capa de comunicación en tiempo real se apoya en Socket.io. A nivel de diseño, el sistema aplica una estructura modular por capas (rutas, controladores, servicios, middlewares y validadores), facilitando la escalabilidad y el mantenimiento.

El proyecto responde a una necesidad real del sector productivo: reducir la distancia entre el talento técnico en formación y las empresas que necesitan perfiles tecnológicos. Para ello, incorpora funcionalidades de perfil profesional, conexión entre usuarios, publicación de contenido y gestión de información de interés para prácticas y empleo.

Además del desarrollo software, el trabajo integra los resultados de aprendizaje de distintos módulos del ciclo DAM: modelado de datos, servicios en red, diseño de interfaces, control de versiones, documentación técnica y criterios de sostenibilidad en infraestructura. La memoria recoge metodología, resultados, limitaciones y líneas futuras, asegurando trazabilidad entre los objetivos definidos y los artefactos desarrollados.

Cuadro 1.1: Ficha técnica del proyecto

Dato	**Información**
Nombre del proyecto	FPConnect
Tipo de aplicación	Web full-stack con backend propio
Ámbito geográfico	Comunidad Autónoma de Andalucía
Usuarios objetivo	Alumnos FP, centros educativos, empresas
Stack principal	React + Vite + Node.js + Express + PostgreSQL + Prisma + Socket.io
Año de desarrollo	2025 – 2026

2. Glosario (ES/EN)

Cuadro 2.1: Términos clave del proyecto

ES	**EN**	**Definición breve**
Autenticación	Authentication	Verificación de identidad del usuario.
Autorización	Authorization	Control de permisos según rol.
ORM	Object-Relational Mapping	Mapeo entre objetos de código y tablas relacionales.
Migración	Migration	Cambio versionado del esquema de base de datos.
Punto final API	API endpoint	Ruta HTTP para consumir una funcionalidad del backend.
Tiempo real	Real-time communication	Intercambio de eventos sin refresco manual.
Despliegue	Deployment	Puesta en producción del sistema.

3. Planteamiento del problema y justificación

3.1. Contexto del problema

La Formación Profesional en España, y en Andalucía en particular, ha experimentado un crecimiento notable en los últimos años tanto en número de alumnos matriculados como en la diversificación de ciclos formativos ofertados. Sin embargo, la transición del mundo educativo al mercado laboral sigue siendo, en muchos casos, un proceso poco transparente y con escasos canales formales de comunicación entre las partes implicadas.

Los alumnos de ciclos como DAM, DAW, ASIR o SMR cuentan con un perfil técnico muy demandado, pero rara vez disponen de una plataforma especializada donde visibilizar su trabajo y conectar directamente con empresas de su entorno. LinkedIn, aunque generalista, no está optimizado para el perfil de un alumno en prácticas o recién graduado de FP. Las bolsas de empleo genéricas tampoco cubren bien las particularidades del sistema de Formación Profesional.

3.2. Motivación del proyecto

La motivación principal detrás de FPConnect nace de la experiencia propia vivida a lo largo del ciclo de DAM: la dificultad de encontrar empresas para realizar las prácticas del módulo de FCT, la falta de visibilidad que tienen los proyectos desarrollados durante el ciclo, y la ausencia de un directorio actualizado de centros FP en Andalucía con información detallada sobre los ciclos que imparten.

Este proyecto intenta ser una respuesta práctica y funcional a esas carencias, aplicando los conocimientos adquiridos durante el ciclo en tecnologías actuales y ampliamente demandadas en el mercado.

4. Fundamentación teórica y estado del arte

El desarrollo de plataformas de empleabilidad educativa se apoya en tres pilares: sistemas de información en red, persistencia de datos relacionales y diseño centrado en usuario. En los últimos años, el ecosistema JavaScript ha consolidado arquitecturas desacopladas frontend-backend, donde React permite construir interfaces componibles y Express facilita la exposición de servicios REST.

En persistencia, PostgreSQL se mantiene como una de las opciones más robustas para aplicaciones transaccionales, mientras que Prisma simplifica la trazabilidad entre modelo lógico y consultas de aplicación. En comunicación en tiempo real, Socket.io aporta una capa de eventos bidireccionales adecuada para mensajería y notificaciones, resolviendo limitaciones de polling tradicional.

Desde la perspectiva funcional, las plataformas generalistas de empleo no cubren completamente las necesidades del contexto FP (prácticas, convenios centro-empresa, visibilidad de proyectos académicos), por lo que existe oportunidad para una solución específica con segmentación por rol y ciclo formativo.

5. Objetivos

1. Diseñar e implementar una plataforma web funcional con tres roles diferenciados: alumnado, centro educativo y empresa.
2. Modelar una base de datos relacional coherente con los procesos de la aplicación, aplicando criterios de integridad y normalización.
3. Implementar un backend con API REST, autenticación segura y comunicación en tiempo real.
4. Construir una interfaz usable y *responsive*, con navegación coherente y componentes reutilizables.
5. Documentar técnicamente el proyecto y su cumplimiento intermodular, incluyendo límites, riesgos y líneas de mejora.

6. Planificación temporal y evaluación de coste

6.1. Planificación temporal

La ejecución del proyecto se ha organizado en fases iterativas para reducir riesgo técnico y facilitar la validación continua con el tutor. Cada fase incluye objetivos, entregables y revisión.

Cuadro 6.1: Cronograma resumido por fases

Fase	Contenido principal	Entregables
Fase 1	Análisis inicial, alcance y definición de arquitectura	Documento de alcance y stack seleccionado
Fase 2	Modelado de datos relacional y estructura de backend	Esquema Prisma, migraciones y endpoints base
Fase 3	Implementación frontend por roles y navegación	Vistas por rol y componentes reutilizables
Fase 4	Integración full-stack y comunicación en tiempo real	API conectada, sockets funcionales y pruebas manuales
Fase 5	Ajustes finales, documentación y preparación de defensa	Memoria, presentación y anexos de cumplimiento

6.2. Evaluación de coste (estimación académica)

El proyecto se ha desarrollado en contexto académico sin coste de licencias propietarias obligatorias, priorizando herramientas *open source*. La siguiente estimación representa un escenario de producción básico para un piloto real.

Cuadro 6.2: Estimación de coste mensual de operación

Concepto	**Descripción**	**Coste estimado/mes**
VPS backend	Servidor Linux con 2 vCPU, 4 GB RAM	20 - 35 EUR
Base de datos	PostgreSQL gestionado o autoalojado en VPS	0 - 30 EUR
Dominio + TLS	Dominio anual prorrateado + certificados Let's Encrypt	1 - 2 EUR
Backups	Almacenamiento incremental para copias diarias	3 - 8 EUR
Monitorización	Logs y alertas básicas	0 - 10 EUR
Total estimado	Operación mínima de piloto	24 - 85 EUR

7. Análisis de requisitos

7.1. Requisitos funcionales priorizados

La priorización se ha realizado por impacto en valor de usuario y dependencia técnica. Se distinguen requisitos de tipo *must-have* (imprescindibles para operación) y *should-have* (evolutivos de alto valor).

Cuadro 7.1: Priorización funcional

Código	**Requisito**	**Prioridad**
RF-01	Registro e inicio de sesión con roles diferenciados	Must-have
RF-02	Gestión de perfil y datos básicos por tipo de usuario	Must-have
RF-03	Publicaciones, comentarios e interacciones sociales	Must-have
RF-04	Conexiones entre usuarios y visibilidad por rol	Should-have
RF-05	Mensajería y eventos en tiempo real	Should-have
RF-06	Recuperación personalizada de contraseña	Should-have

7.2. Requisitos técnicos de seguridad y acceso

- Validación estricta de entrada en backend (Joi) para reducir errores de formato y abuso.
- Autenticación con JWT y renovación controlada con refresh token.
- Contraseñas protegidas mediante hash con bcrypt y política mínima de complejidad.
- Gestión de errores centralizada para no exponer información sensible del sistema.
- Control de acceso por rol en rutas privadas y acciones sensibles.

8. Análisis del Sistema

8.1. Identificación de actores

Se han identificado tres tipos de usuarios principales que interactúan con la plataforma, cada uno con necesidades y funcionalidades específicas:

Cuadro 8.1: Actores del sistema

Actor	Descripción	Necesidades principales
Alumno	Estudiante de un ciclo formativo de FP en Andalucía	Visibilizar su perfil, encontrar empresas y centros, acceder a noticias del sector
Centro FP	Instituto o centro educativo que imparte ciclos formativos	Gestionar su comunidad de alumnos, conectar con empresas para convenios de FCT
Empresa	Empresa que busca incorporar perfiles técnicos de FP	Buscar y filtrar talento, publicar vacantes, establecer contacto con centros

8.2. Requisitos funcionales

8.2.1. Módulo de autenticación y gestión de roles

- El sistema debe permitir el registro e inicio de sesión de usuarios diferenciando entre los tres roles disponibles.
- Cada rol debe redirigir al usuario a su *dashboard* correspondiente tras la autenticación.
- Las rutas y funcionalidades deben estar protegidas según el rol del usuario autenticado.

8.2.2. Funcionalidades del Alumno

- Acceso a un *dashboard* personal con estadísticas de perfil y noticias del sector.

- Directorio de centros FP con filtros por ciclo formativo y provincia.
- Visualización de centros y empresas sobre un mapa interactivo de Andalucía.
- Acceso a perfiles públicos de otros alumnos de la plataforma.
- Perfil público propio con información de CV, habilidades técnicas, proyectos y experiencia.
- *Feed* de noticias sobre becas, convocatorias y eventos relevantes para FP.

8.2.3. Funcionalidades del Centro FP

- *Dashboard* con métricas del centro: número de alumnos registrados, empresas colaboradoras, ciclos impartidos.
- Listado y gestión de alumnos registrados en el centro.
- Visualización de empresas colaboradoras actuales y potenciales.
- Perfil público del centro con información de contacto, ciclos y ubicación.

8.2.4. Funcionalidades de la Empresa

- Buscador de talento con filtros por ciclo formativo, ciudad y disponibilidad.
- Acceso al perfil completo de cada alumno, incluyendo *skills* y proyectos.
- Gestión de vacantes de prácticas (FCT) y de empleo.
- Directorio de centros FP para establecer convenios de colaboración.
- Perfil público de empresa con información corporativa.

8.3. Requisitos no funcionales

Cuadro 8.2: Requisitos no funcionales

Requisito	**Descripción**
Usabilidad	La interfaz debe ser intuitiva y consistente. Un usuario nuevo debe poder orientarse sin manual.
Rendimiento	La aplicación web debe cargar en menos de 3 segundos en condiciones normales de red.
Multiplataforma	El mismo código base debe funcionar como aplicación web y como aplicación de escritorio nativa.
Escalabilidad	La arquitectura backend basada en Node.js, PostgreSQL y Prisma debe soportar el crecimiento en número de usuarios sin cambios estructurales en el frontend.
Seguridad	La autenticación debe basarse en JWT. Los datos sensibles no deben exponerse en el frontend.
Mantenibilidad	El código debe seguir una estructura modular y estar organizado por componentes reutilizables.

9. Diseño del Sistema

9.1. Arquitectura general

FPConnect sigue una arquitectura cliente-servidor desacoplada. El frontend, desarrollado con React y Vite, consume una API REST construida con Node.js y Express. El backend integra autenticación por JWT, validación de entrada, acceso a datos mediante Prisma ORM sobre PostgreSQL y servicios en tiempo real con Socket.io.

La estructura del backend se organiza por capas (rutas, controladores, servicios, middlewares, validadores), lo que permite separar responsabilidades y facilitar el mantenimiento evolutivo.

Cuadro 9.1: Capas tecnológicas de la arquitectura

Capa	**Tecnología**	**Responsabilidad**
Frontend web	React + Vite	Interfaz de usuario, navegación y estado cliente
Backend API	Node.js + Express	Lógica de negocio y exposición de servicios REST
Datos	PostgreSQL + Prisma ORM	Persistencia, relaciones y migraciones
Tiempo real	Socket.io	Eventos y mensajería bidireccional
Mapas	Leaflet.js + OpenStreetMap	Visualización geográfica de centros y empresas
Despliegue	Linux Server + Nginx + PM2	Ejecución de backend en entorno productivo

9.2. Estructura de directorios

El proyecto sigue una organización modular basada en la separación por responsabilidades:

Cuadro 9.2: Estructura de directorios del proyecto

Ruta	**Contenido**
`src/components/UI.jsx`	Componentes reutilizables: Badge, Avatar, Card, NavBar, etc.
`src/data/datos.js`	Datos de prueba realistas: centros, alumnos, empresas, noticias
`src/pages/LoginPage.jsx`	Pantalla de autenticación con selección de rol
`src/pages/AlumnoApp.jsx`	Dashboard completo del perfil de alumno
`src/pages/CentroApp.jsx`	Dashboard del centro educativo
`src/pages/EmpresaApp.jsx`	Dashboard de la empresa con buscador de talento
`src/styles/global.css`	Reset CSS y estilos globales compartidos
`src/App.jsx`	Router principal con enrutado por rol
`src/main.jsx`	Punto de entrada de la aplicación React
`vite.config.js`	Configuración del bundler Vite

9.3. Modelo de datos

El modelo de datos está diseñado para su implementación en PostgreSQL mediante Prisma ORM. Se han identificado las siguientes entidades principales y sus relaciones:

Cuadro 9.3: Entidades del modelo de datos

Tabla	Campos principales	Relaciones
`usuarios`	id, email, rol, created_at	Base de todos los perfiles
`alumnos`	id, usuario_id, nombre, ciclo, centro_id, ciudad, bio, nota, disponible, cv_url, foto_url	N:1 con usuarios, N:1 con centros, N:M con skills
`centros`	id, usuario_id, nombre, ciudad, provincia, descripcion, telefono, email, web, lat, lng	N:1 con usuarios, N:M con ciclos
`empresas`	id, usuario_id, nombre, sector, ciudad, descripcion, empleados, logo_url	N:1 con usuarios
`vacantes`	id, empresa_id, titulo, tipo, ciclo, estado, created_at	N:1 con empresas
`noticias`	id, titulo, categoria, resumen, fecha, urgente, url	Entidad independiente
`intereses`	id, empresa_id, alumno_id, created_at	N:M entre empresas y alumnos
`centros_ciclos`	centro_id, ciclo	N:M entre centros y ciclos
`alumnos_skills`	alumno_id, skill	N:M entre alumnos y habilidades

9.4. Diseño de la interfaz de usuario

La interfaz de FPConnect sigue los principios de diseño limpio y moderno, con una paleta de colores basada en verdes y blancos que transmite frescura y profesionalidad. Se han aplicado los siguientes criterios de diseño:

- **Consistencia visual:** todos los *dashboards* comparten los mismos componentes base (Card, Badge, Avatar, NavBar) adaptados a cada rol.
- **Responsive design:** la aplicación está diseñada para funcionar correctamente tanto en escritorio como en dispositivos móviles.
- **Tipografía:** se utiliza *DM Sans* de Google Fonts para toda la interfaz, una fuente moderna y de alta legibilidad.

- **Componentes reutilizables:** la arquitectura de componentes en React permite que elementos como las tarjetas de perfil o las insignias de ciclo sean reutilizados en todos los *dashboards*.

10. Metodología

La metodología de trabajo ha combinado desarrollo incremental y validación continua, organizada en iteraciones cortas:

1. Definición de alcance funcional y diseño preliminar de datos.
2. Implementación de base de proyecto (frontend, backend, autenticación y estructura modular).
3. Desarrollo de funcionalidades por dominio (usuarios, publicaciones, comentarios y conexiones).
4. Integración de comunicación en tiempo real y pruebas manuales de flujos críticos.
5. Documentación técnica y mapeo de cumplimiento respecto a criterios intermodulares.

Para el control de calidad se han aplicado validaciones de entrada, separación de lógica por servicios, uso de migraciones versionadas y comprobaciones funcionales en entorno local.

11. Codificación

11.1. Entorno de programación y herramientas

El desarrollo se ha realizado en Visual Studio Code con control de versiones Git, Node.js LTS y gestión de dependencias mediante npm. Para persistencia se utiliza Prisma sobre PostgreSQL y para comunicación en tiempo real Socket.io.

11.2. Aspectos relevantes de implementación

11.2.1. Validación de datos

Toda petición de entrada en rutas críticas se valida en middleware con esquemas explícitos, interrumpiendo el flujo cuando la carga útil no cumple formato o reglas de negocio básicas.

11.2.2. Control de acceso

Las rutas privadas exigen token válido y, en funcionalidades concretas, se verifica rol y propiedad del recurso para evitar accesos no autorizados.

11.2.3. Protección de la información

No se almacenan contraseñas en claro. Los secretos de entorno se externalizan en variables de configuración y no se incluyen en repositorio público.

11.2.4. Estructura modular del backend

La lógica se reparte por responsabilidades: rutas para entrada HTTP, controladores para coordinación, servicios para reglas de negocio y acceso a datos, y middlewares para capas transversales.

12. Resultados

12.1. Estado actual del desarrollo

En la versión actual del repositorio se encuentra implementada una base funcional del sistema con autenticación, backend operativo, persistencia relacional y frontend por roles. Se han validado los flujos principales de acceso, consulta y publicación.

Cuadro 12.1: Estado de implementación – Fase 1

Funcionalidad	**Descripción**
Login multi-rol	Pantalla de inicio de sesión con selección de rol (alumno, centro, empresa).
Dashboard alumno	Estadísticas, noticias, directorio de centros con filtros, mapa, comunidad y perfil.
Dashboard centro	Métricas, gestión de alumnos, empresas colaboradoras y perfil del centro.
Dashboard empresa	Buscador de talento con filtros, gestión de vacantes y directorio de centros.
Datos y persistencia	Modelo de datos relacional con Prisma y migraciones versionadas.
Backend API	API REST con Express, validación Joi y autenticación JWT + refresh token.
Mapa interactivo	Geolocalización real de centros y empresas con Leaflet.js.
Mensajería	Sistema de mensajería entre empresas y alumnos.
Recuperación de contraseña	Flujo personalizado de recuperación con pregunta/respuesta de seguridad.
Tiempo real	Comunicación simultánea cliente-servidor mediante Socket.io.

13. Pruebas de ejecución

13.1. Pruebas funcionales

Se han probado manualmente los recorridos críticos: autenticación, navegación por rol, publicaciones, comentarios, conexiones y recuperación de contraseña. También se validó respuesta de endpoints con peticiones HTTP controladas.

13.2. Pruebas de usabilidad

La navegación se revisó para minimizar fricción en tareas frecuentes: iniciar sesión, localizar información principal, publicar contenido y consultar perfiles.

13.3. Pruebas de accesibilidad

Se ha mantenido contraste visual adecuado en elementos principales, tipografía legible y estructura de interacción consistente. Como línea de mejora se plantea auditoría formal con herramientas especializadas (por ejemplo TAW o Lighthouse).

13.4. Pruebas de seguridad

Se verificaron casos básicos de denegación por credenciales inválidas, tokens caducados y payloads no válidos. La recuperación de contraseña exige verificación previa y actualización segura del hash.

13.5. Pruebas de carga (estado actual)

No se han ejecutado pruebas de carga automatizadas extensivas en esta fase. Se incluye como mejora prioritaria incorporar escenarios de concurrencia y umbrales de rendimiento medibles.

14. Manuales de usuario

14.1. Objetivo

Facilitar a usuarios no técnicos una guía mínima de operación para comenzar a usar la plataforma y resolver incidencias comunes.

14.2. Requisitos e instalación

- Frontend: `npm install` y `npm run dev` en la raíz del proyecto.
- Backend: `npm install` en `backend/`, base de datos PostgreSQL operativa y `npm run dev`.
- Navegador moderno actualizado para uso del cliente web.

14.3. Operaciones habituales

1. Registro o inicio de sesión con credenciales válidas.
2. Selección de rol y acceso al panel correspondiente.
3. Gestión de perfil y consulta de información por dominio (alumno, centro o empresa).
4. Interacción social mediante publicaciones, comentarios y conexiones.

14.4. Mensajes de error comunes

- Credenciales inválidas: revisar email/contraseña o usar recuperación.
- Token expirado: repetir autenticación o renovar sesión.
- Error de validación: corregir campos marcados y reenviar formulario.

15. Conclusiones

15.1. Grado de consecución de objetivos

El proyecto alcanza de forma satisfactoria los objetivos principales definidos: arquitectura full-stack operativa, modelo relacional implementado, servicios de autenticación y capa de interfaz por roles. Además, se han integrado funcionalidades de valor añadido como comunicación en tiempo real y recuperación de contraseña personalizada.

15.2. Limitaciones

Las limitaciones más relevantes han sido el tiempo disponible para ampliar cobertura de pruebas automatizadas y la falta de empaquetado móvil nativo en esta fase. También existen mejoras pendientes en observabilidad avanzada y escalado de infraestructura para alta concurrencia.

15.3. Futuras líneas de trabajo

- Implementar cliente móvil nativo (React Native o Flutter) reutilizando la API existente.
- Incorporar analítica de uso y cuadros de mando para centros y empresas.
- Añadir CI/CD completo con ejecución automática de tests y despliegue controlado.
- Fortalecer seguridad con rate limiting y políticas avanzadas de gestión de sesiones.

16. Bibliografía

- PostgreSQL Global Development Group. (2025). *PostgreSQL Documentation*. https://www.postgresql.org/docs/
- Prisma Labs. (2026). *Prisma ORM Documentation*. https://www.prisma.io/docs
- OpenJS Foundation. (2026). *Node.js Documentation*. https://nodejs.org/docs
- Express.js. (2026). *Express Web Framework Documentation*. https://expressjs.com/
- React Team. (2026). *React Documentation*. https://react.dev/
- Socket.IO Authors. (2026). *Socket.IO Documentation*. https://socket.io/docs/

17. Anexos

- Anexo A: Matriz de cumplimiento intermodular: `Requisitos/MATRIZ_CUMPLIMIENTO_INTERMODULAR.md`
- Anexo B: Configuración estructurada del proyecto (YAML): `Requisitos/config_proyecto.yaml`
- Anexo C: Plan de despliegue Linux: `Requisitos/despliegue_produccion_linux.md`
- Anexo D: Plan de sostenibilidad: `Requisitos/plan_sostenibilidad_cpd.md`
- Anexo E: Resumen ejecutivo en inglés y glosario técnico bilingüe.